

Course Grade and Productivity Application System Design Document

System Design Document

ELEC 376

Department of Electrical and Computer Engineering

Faculty of Engineering and Applied Science Queen's University

Prepared by Group 8

Ashton Neto 20385873

Clarke Needles 20424709

Dayna Dranitsaris 20396159

Fedor Kim 20414436

Nikki Scharlach 20224068

Yehia Nouredin 20391895

Teaching Assistant

Tooraj Taraz

10/15/25

Table of Contents

1. System Purpose.....	1
2. Design Goals.....	1
2.1 Performance.....	1
2.2 Dependability	1
2.3 Cost	2
2.4 Maintenance	2
3. End User Criteria	2
4. Definitions, Acronyms, and Abbreviations.....	3
5. Use Cases	3
5.1 Case 1: Track Personal Academics	3
5.1.1 Participating Actors.....	3
5.1.2 Flow of Events	3
5.1.3 Entry Condition	4
5.1.4 Exit Condition	4
5.1.5 Quality Requirements	4
5.2 Case 2: Compete Against Friends.....	4
5.2.1 Participating Actors.....	4
5.2.2 Flow of Events	4
5.2.3 Entry Condition	5
5.2.4 Exit Condition	5
5.2.5 Quality Requirements	5
6. Object Model	5
7. Sequence Diagrams.....	7
8. Class Diagram	9
9. State Chart Diagrams	10

10. Use Case Diagrams	11
11. System Architecture	11
11.1 Overview	11
11.2 Subsystem Decomposition	11
11.3 Software/Hardware Mapping	12
11.4 Persistent Data Management	13
11.5 Access Control and Security	13
11.6 Global Software Control	14
11.7 Boundary Conditions	14
12. GUI	14
13. Project Timeline: Gantt Chart	17

Table of Figures

Figure 1: Sequence diagram for the first use case	7
Figure 2: Sequence diagram for the second use case	8
Figure 3: Class Diagram	9
Figure 4: State chart diagram for course page view.	10
Figure 5: State chart diagram for friend page view.	10
Figure 6: Use case diagram	11
Figure 7: Database structure	13
Figure 8: Current homepage window	14
Figure 9: Window shown after clicking an individual course	15
Figure 10: Window shown after clicking on a friend	16
Figure 11: The window shown after clicking 'Courses' from the homepage	17
Figure 12: Gantt chart showing the completion of the project so far and projected timeline	18

1. System Purpose

An overlooked problem that Queen's students currently face is the lack of easy access to their grades throughout any given semester. How grades are displayed on any given OnQ course fully depends on the professor, and most don't even update the grades tab provided. Additionally, SOLUS only provides letter grades at the end of a semester, when it is too late for students to change their trajectory. This leaves options such as meticulously calculating each given grade and its weight or just disregarding grades until it is too late. The solution provided by the team aims to compile all grades in one centralized application. Users will still have to input grades and weighting manually, but the program will automatically calculate overall grades and keep everything together. To further incentivize use the process will be gamified with ranks assigned based on factors such as grades and logged study hours. Friendly competition leaderboards will let students see their standing against selected peers and push them to do better. Such a solution will make it significantly easier for students to see their grades and improve them as opposed to the options currently available.

2. Design Goals

2.1 Performance

After a user produces input (button click, text input) the program should respond appropriately within 2 seconds maximum. When the user adds a course or a friend, it should take a maximum of 1 second to reflect on the courses/friends list. When a user reaches a new rank, it should be reflected in their status within 3 seconds maximum. The program should not take up an unreasonable amount of space on the hardware it is being run on.

2.2 Dependability

Users will have the ability to sign up and log in to run their own instance of the program at the same time as others, interacting through button and keyboard inputs. Exceptions will handle cases where fields for email and password receive invalid input. They will also handle if a user attempts to add a course or friend more than once. Testing will be done to ensure that program behavior and output is as expected and correct with respect to the code.

2.3 Cost

The program will run on a computer as a web application, with no associated development cost. There is also no costs associated with upgrades, maintenance, or administration to meet the needs of those on a student budget. Upgrades and maintenance of the app would require it to be pushed to the user computer again.

2.4 Maintenance

Connections between the front-end Qt **ui.** files with the backend will make each functionality easy to map to specific requirements. This modular structure allows for straightforward integration of new features; consider adding a function to the backend and linking it to the Qt interface using a button, etc. The web application is being developed using Qt and Visual Studio Code and will be deployable on any PC or desktop. Maintenance will be performed with regular updates and test runs of the code and the most recent version of the application can be pulled to receive potential expansions such as a global leaderboard.

3. End User Criteria

The GUI will be user friendly to ensure the application is intuitive and aesthetically pleasing. The user will be able to sign up and then log in to their profile and be brought to a main screen with their rank. This screen will also display their list of courses and friends as well as the ability to add new ones. It will also include buttons at the bottom to start a study timer, log study hours, and input a new grade. Finally, it will have two additional buttons to log out (bring them back to the sign up/log in screen) and get help (bring them to a list of FAQ). If a user clicks on an individual course, they will be brought to a page with their rank in that course, completed study hours, and current grade (including grade distribution based on their input grades and editing of the course). If a user clicks on an individual friend, they are brought to a page with that friend's rank, total study hours, and active courses along with their rank logo in each course. If a user clicks on the courses tab, they can see all of their courses (current and past) along with their grade in each, their career rank, average grade, and statistics listed on the right side (longest study session, highest/lowest grades, etc.)

4. Definitions, Acronyms, and Abbreviations

The definitions and acronyms used in this report are outlined for better understanding below.

GUI: Graphical User Interface

Qt: System for GUI development

UI: User Interface

SOLUS: Student On Line University System (Queen's University student information portal)

OnQ: Queen's course management platform based on D2L platform

GPA: Grade Point Average

5. Use Cases

The following section outlines cases such as tracking academics and competing with peers and how the user will interact with the program.

5.1 Case 1: Track Personal Academics

Tracking personal academics is how the user tracks progress for any given course and how this logging influences their standing for it.

5.1.1 Participating Actors

- User
- Courses class
- Assignments (courses substructure)
- Database class

5.1.2 Flow of Events

1. The user wants to know their overall grade for a course
2. The user clicks on their course.
3. The data is taken from the database class and populates the course page.
4. The user clicks on "add assignment" and enters the assignment name, grade earned (0 to 100), and weighting (0 to 100).

5. The assignments structure automatically calculates and updates the overall course grade for the user to see.
6. The assignment is added to the database.
7. The user wants to see their rank in the course.
8. The user clicks log study hours for the added assignment and adds a number for hours studied.
9. The study hours field updates in the database.
10. The course structure calculates rank based on overall grade and study hours, displaying for the user to see.
11. The rank for the user is updated on the database.
12. The user understands their grades for the course and has earned an achievement for their efforts.

5.1.3 Entry Condition

The user has an account, is logged into the application, and has added a course to their profile.

5.1.4 Exit Condition

The user understands their overall grade for a course as well as their rank in it.

5.1.5 Quality Requirements

The user grades are calculated accurately and immediately upon updating assignments. The user rank is calculated immediately and in a logical manner based on grades and study hours. Input information is easy to view, understand, and edit and is saved properly for the user to access later.

5.2 Case 2: Compete Against Friends

Competing against friends is how the user adds friends and compares their scores with friends.

5.2.1 Participating Actors

- User
- User friends
- Courses class

5.2.2 Flow of Events

1. The user wants to add their friends.
2. The user clicks on add friend and enters their unique username.

3. The leaderboard updates based on the average of course rank scores.
4. The user wants to see the grades their friend earned.
5. The user clicks on their friend in the menu.
6. The user is taken to the page of their friend and can see their assignments and grades for their courses.

5.2.3 Entry Condition

The user has an account and is logged into the application.

5.2.4 Exit Condition

The user has added their friends and can see how well their friends are doing in their courses.

5.2.5 Quality Requirements

The user can add friends and have their leaderboard update immediately and accurately. The friends menu and leaderboard is easy to navigate and understand. When refreshed the leaderboard updates accurately based on changes made by friends to their courses (changes which influence their overall standing).

6. Object Model

1. **Entities:** User, Course, Assignment, StudySession, Friend, Rank, Database
2. **Controls:** LoginControl, SignupControl, CourseControl, RankControl, FriendsControl, StudySessionControl, ErrorControl
3. **Boundaries:**
 - a. AuthMenuButton
 - i. LoginPopUpButton
 - ii. LoginPopUpWindow
 - iii. PopUpWindowCloseButton
 - iv. SignupPopUpButton
 - v. SignupPopUpWindow
 - vi. PopUpWindowCloseButton
 - b. HomeMenuButton

- i. CoursesPopUpButton
- ii. CoursesPopUpWindow
- iii. PopUpWindowCloseButton
- iv. FriendsPopUpButton
- v. FriendsPopUpWindow
- vi. PopUpWindowCloseButton
- vii. InputNewGradePopUpButton
- viii. InputNewGradePopUpWindow
- ix. PopUpWindowCloseButton
- x. StudyTimerPopUpButton
- xi. StudyTimerPopUpWindow
- xii. PopUpWindowCloseButton
- xiii. LogStudyHoursPopUpButton
- xiv. LogStudyHoursPopUpWindow
- xv. PopUpWindowCloseButton
- xvi. PopUpWindowCloseButton
- xvii. FAQmenuButton
- xviii. FAQpopUpButton
- xix. FAQpopUpWindow
- xx. PopUpWindowCloseButton
- c. CourseMenuButton
 - i. AddCoursePopUpButton
 - ii. AddCoursePopUpWindow
 - iii. PopUpWindowCloseButton
 - iv. CourseDetailsPopUpButton
 - v. CourseDetailsPopUpWindow
 - vi. PopUpWindowCloseButton
 - vii. CompleteCoursePopUpButton
 - viii. CompleteCoursePopUpWindow
 - ix. PopUpWindowCloseButton
- d. AssignmentMenuButton
 - i. AddAssignmentPopUpButton

- ii. AddAssignmentPopUpWindow
- iii. PopUpWindowCloseButton
- e. FriendMenuButton
 - i. AddFriendPopUpButton
 - ii. AddFriendPopUpWindow
 - iii. PopUpWindowCloseButton
 - iv. FriendDetailsPopUpButton
 - v. FriendDetailsPopUpWindow
 - vi. PopUpWindowCloseButton

7. Sequence Diagrams

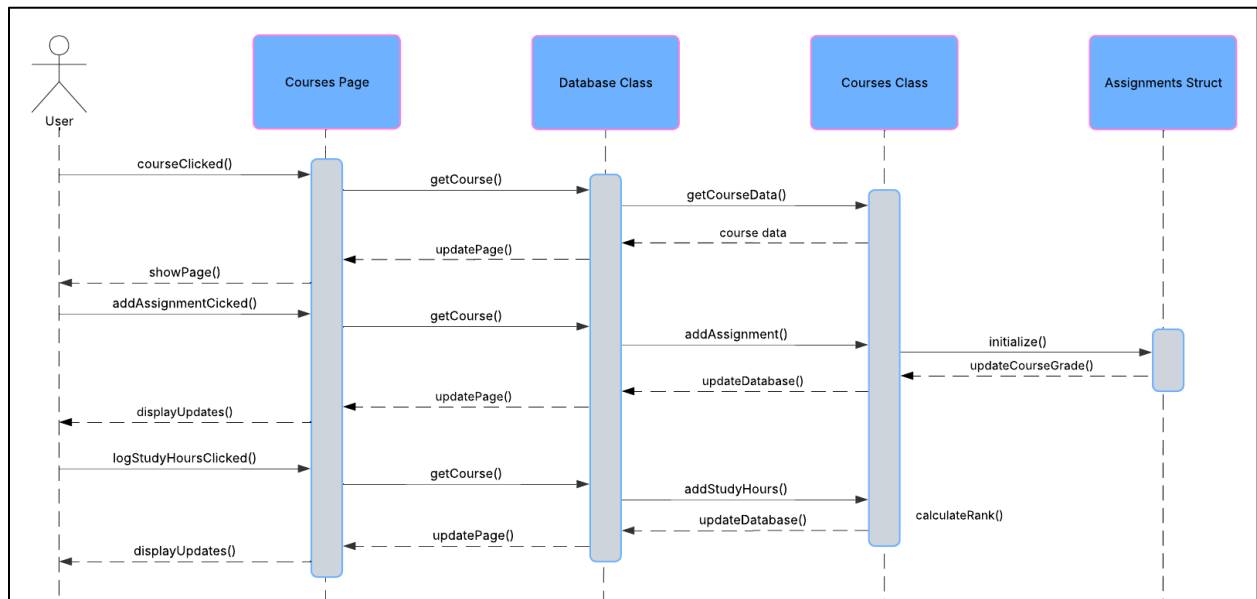


Figure 1: Sequence diagram for the first use case

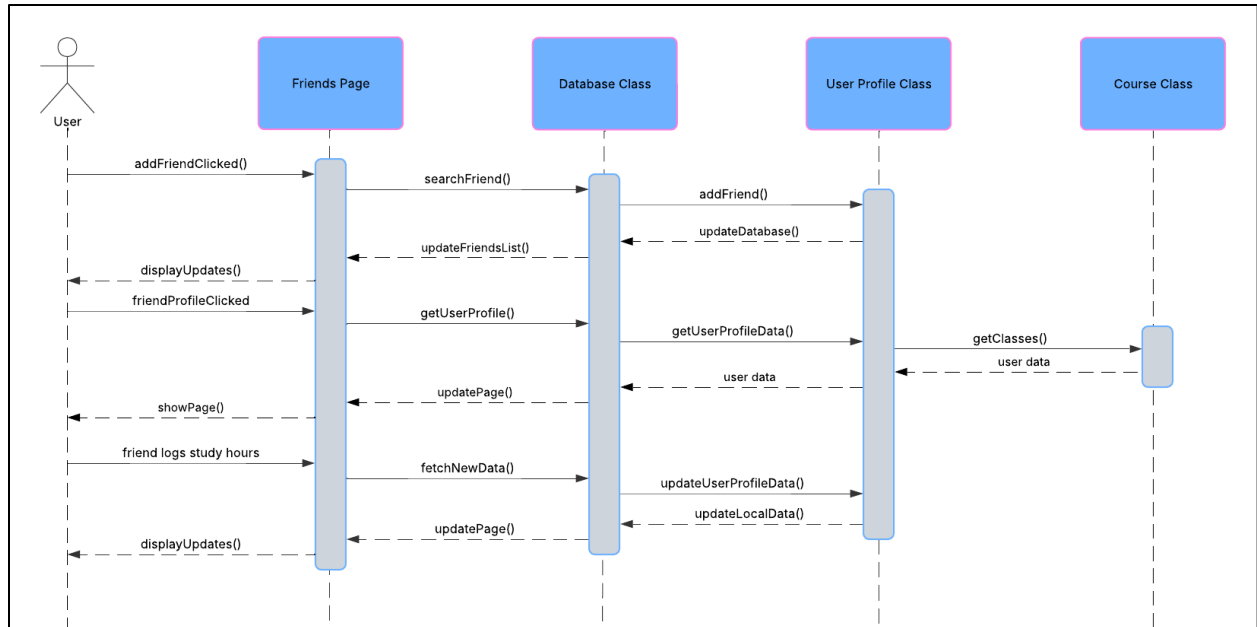


Figure 2: Sequence diagram for the second use case

8. Class Diagrams

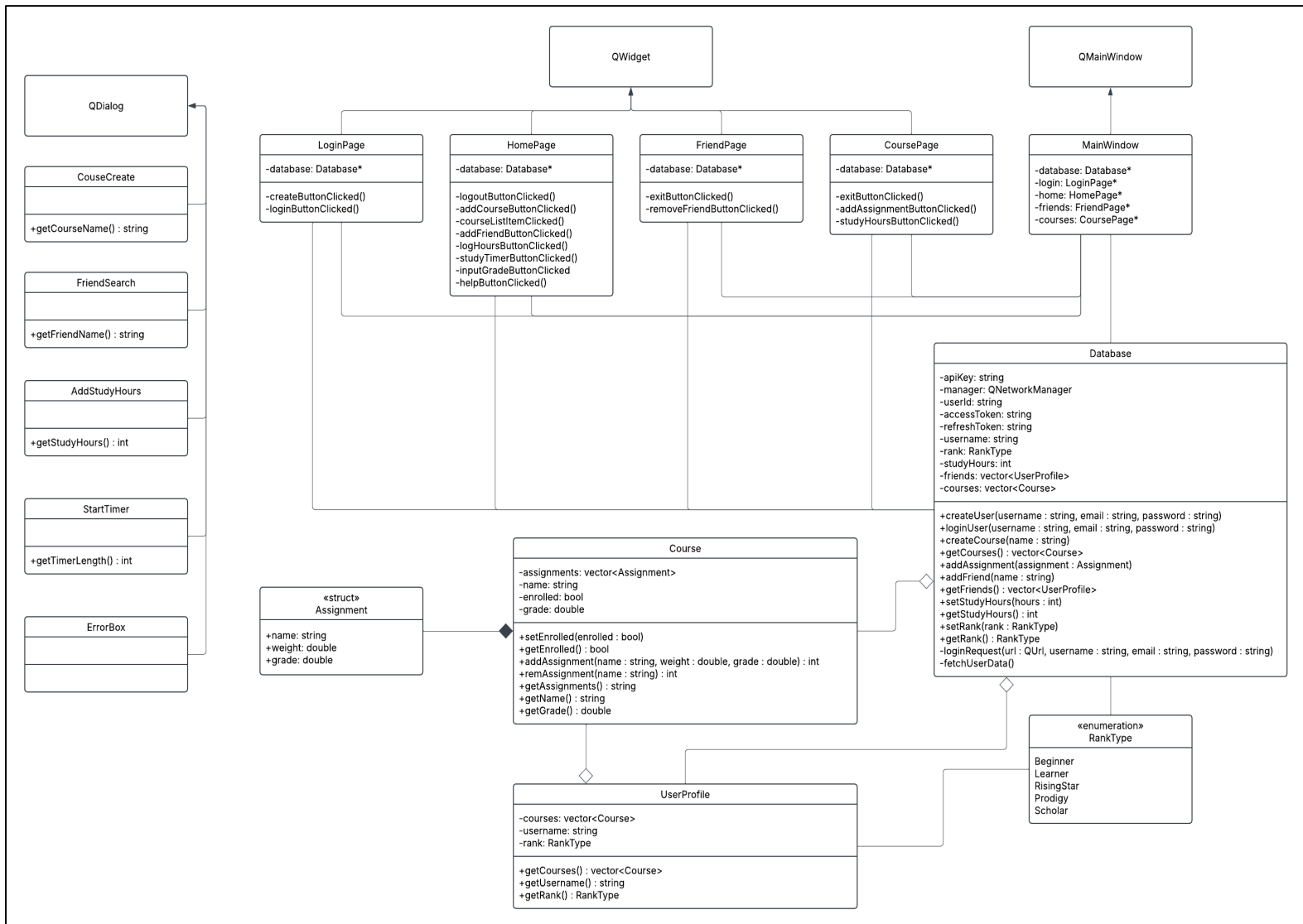


Figure 3: Class Diagram

9. State Chart Diagrams

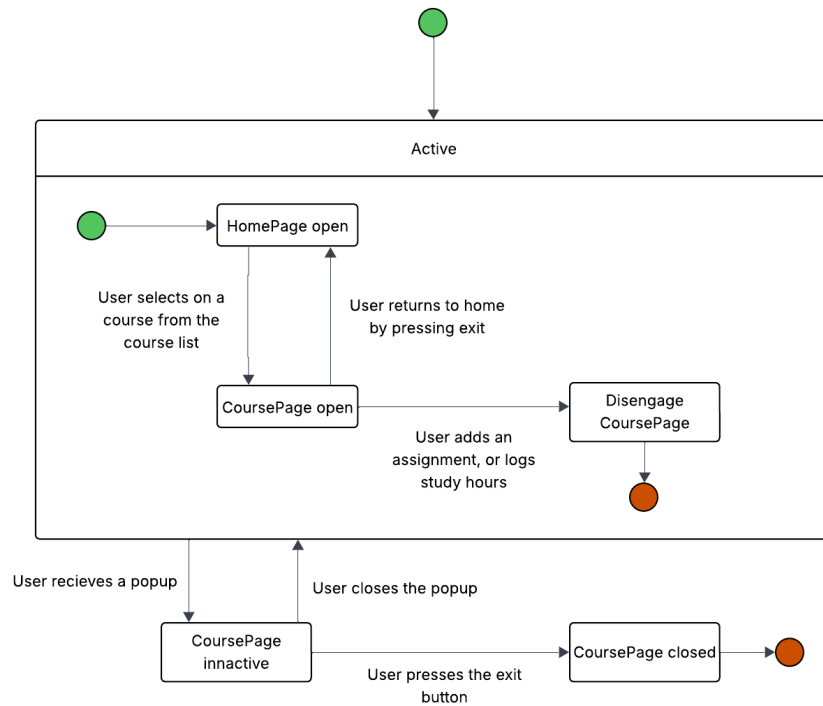


Figure 4: State chart diagram for course page view

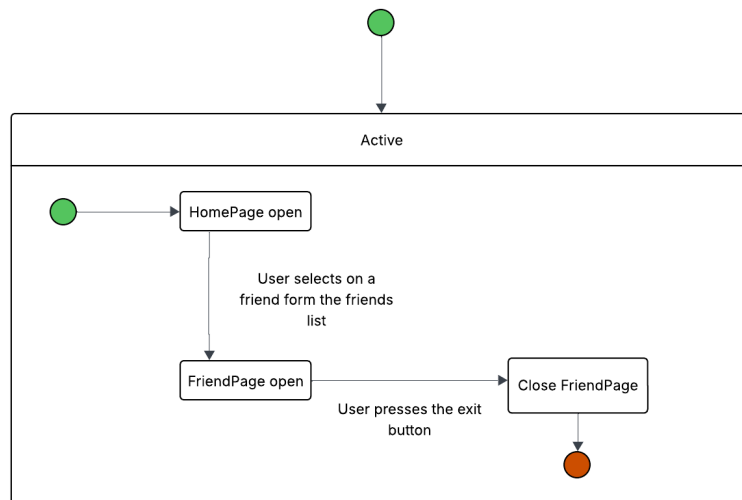


Figure 5: State chart diagram for friend page view

10. Use Case Diagrams

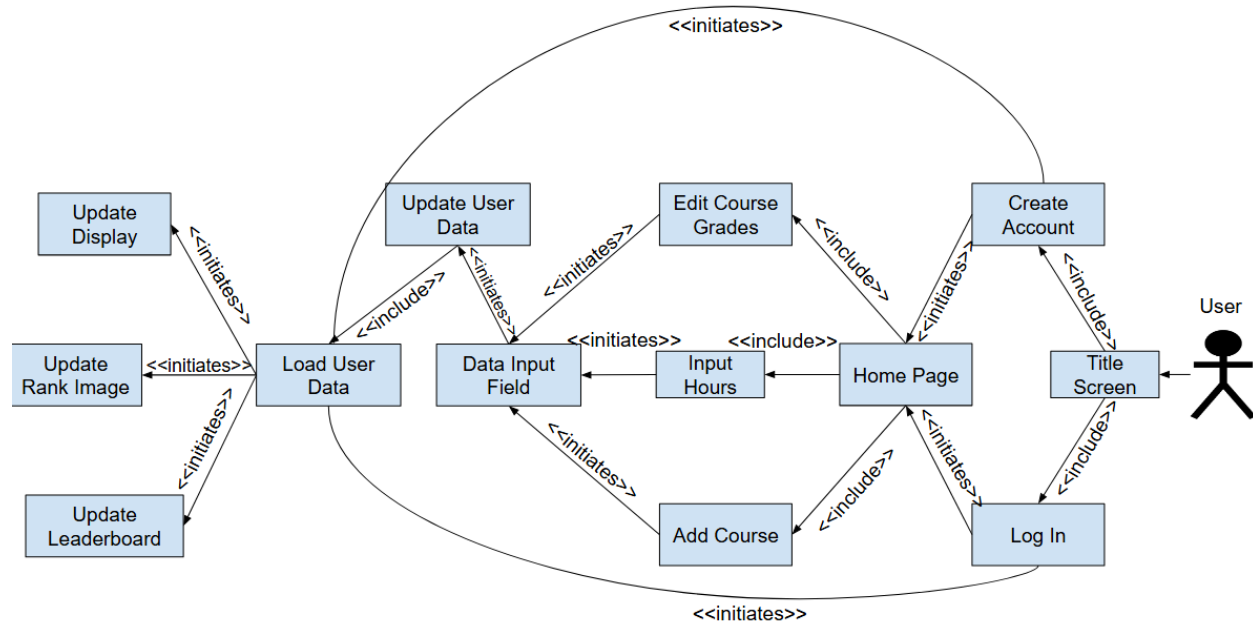


Figure 6: Use case diagram

11. System Architecture

11.1 Overview

The system architecture of this program is comprised of 3 main subsystems. The graphical user interface (GUI), which will take user input through an input field to update the database subsystem, which will then allow the rank subsystem to read from the database and compute a user's rank and grade, as well as change the current rank icon in the GUI. The program will require an internet connection to access the database and will only contain one user mode for the account that is currently logged in.

11.2 Subsystem Decomposition

The application comprises the following four subsystems:

1. Graphical User Interface
2. Database
3. Rank Computation

Each of these subsystems is essential to the proper functionality of the application.

The Graphical User Interface subsystem will be the only subsystem that the user directly interacts with. The subsystem is responsible for providing the user with buttons which trigger new GUI features that the user may interact with further. The coupling of this subsystem is strong as any GUI feature that takes user input will directly impact other subsystems that need to receive this input and correctly handle it. There is strong cohesion as each element in the GUI is single purpose and only perform one task per element as opposed to having multiple ways to achieve the same outcome.

The Database subsystem is the foundation of the program as it is responsible for storing user data, accessing it, and being able to edit the data whenever necessary, and this is done using the online database called Supabase. The stored information is volatile and may change at any time due to users adding or removing accounts or specific data points such as grades. There is strong coupling between this subsystem, the GUI and the rank computation subsystem, as any input to the database is received through the GUI via user inputs, and this data is also essential to computing a user's rank. There is strong cohesion in this subsystem as well as each method that accesses the database is only controlled within this scope through multiple accessors and mutators for specific data points.

The Rank Computation subsystem is not accessed by the user and relies on the database subsystem to provide data that will be processed and displayed on the GUI. There is moderate coupling between this subsystem and the GUI as the data will be processed then displayed on the GUI. The system then has strong coupling with the database as the data computed in this subsystem relies on user data. This subsystem has weak cohesion as it relies on methods from the database subsystem to gather data before computation.

11.3 Software/Hardware Mapping

This application does not map to any hardware or physical components; therefore, this section is not applicable.

11.4 Persistent Data Management

The user of an external database is a key feature of this application. The database used is called Supabase and automatically encrypts user data that is stored, and the data can be accessed through the program as long as there is a viable internet connection. The data is stored permanently unless the database is destroyed. The structure of the database is shown below.

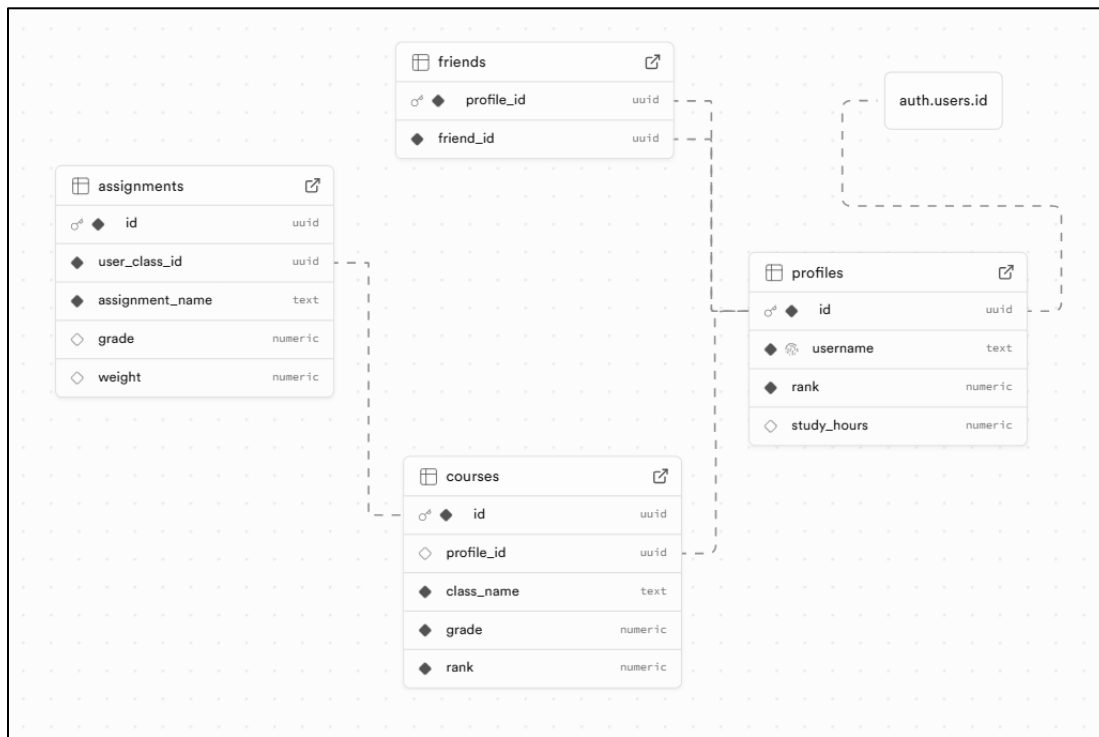


Figure 7: Database structure

11.5 Access Control and Security

	Data Input-Fields	Leader Board	Courses
User	Read / Write	Read	Read / Write

The matrix above explains the permissions of a user for this program. The authentication for this user would happen through password authentication; all encryption and keys are handled automatically by the external database software.

11.6 Global Software Control

Having subsystems work together will not be an issue as each subsystem works individually and there will not be any parallel processing to handle as any chain of methods would be sequential. Most subsystems should process information fast enough as to not impact the user experience. The only drawback being the user's internet connection as slower speeds may impact data transfer times which could cause the user to have to wait if they have an unstable connection.

11.7 Boundary Conditions

If a user attempts to log into an account that is not registered in the online database, they will get an error message notifying them. If the user has very weak or no internet connection the database will return an error message relating to the request which will then be passed onto the user as a popup notification.

12. GUI

Currently Only the login and homepage have been implemented in QT while the backend framework is being developed. However, the goal is to design an app with full functionality for a fun user experience. The current homepage is shown below (Figure 2).

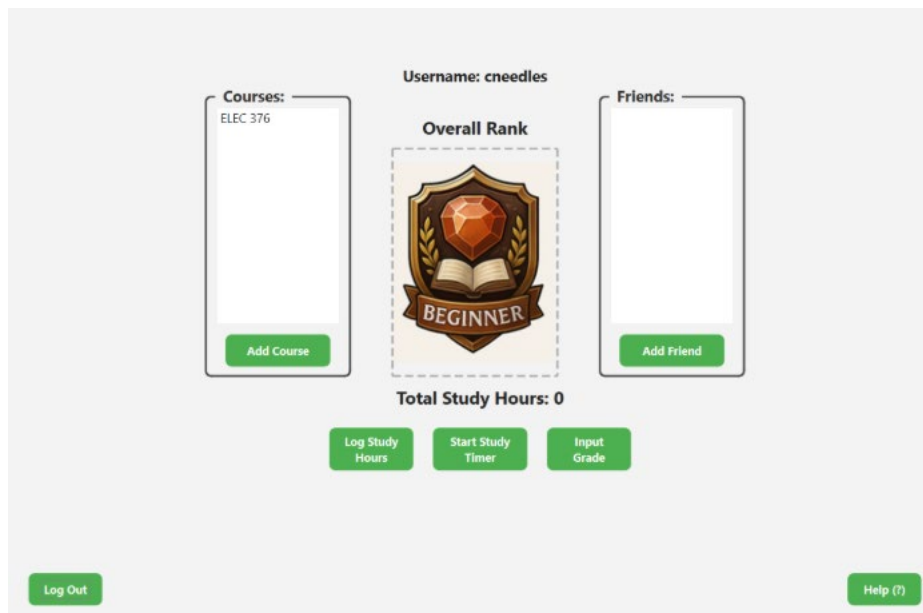


Figure 8: Current homepage window

The team aims to add course, ranks, and friends functionality as seen below. Clicking on a course from the homepage shown above should bring the user to a window in which they can see information and progress in that course (Figure 3).

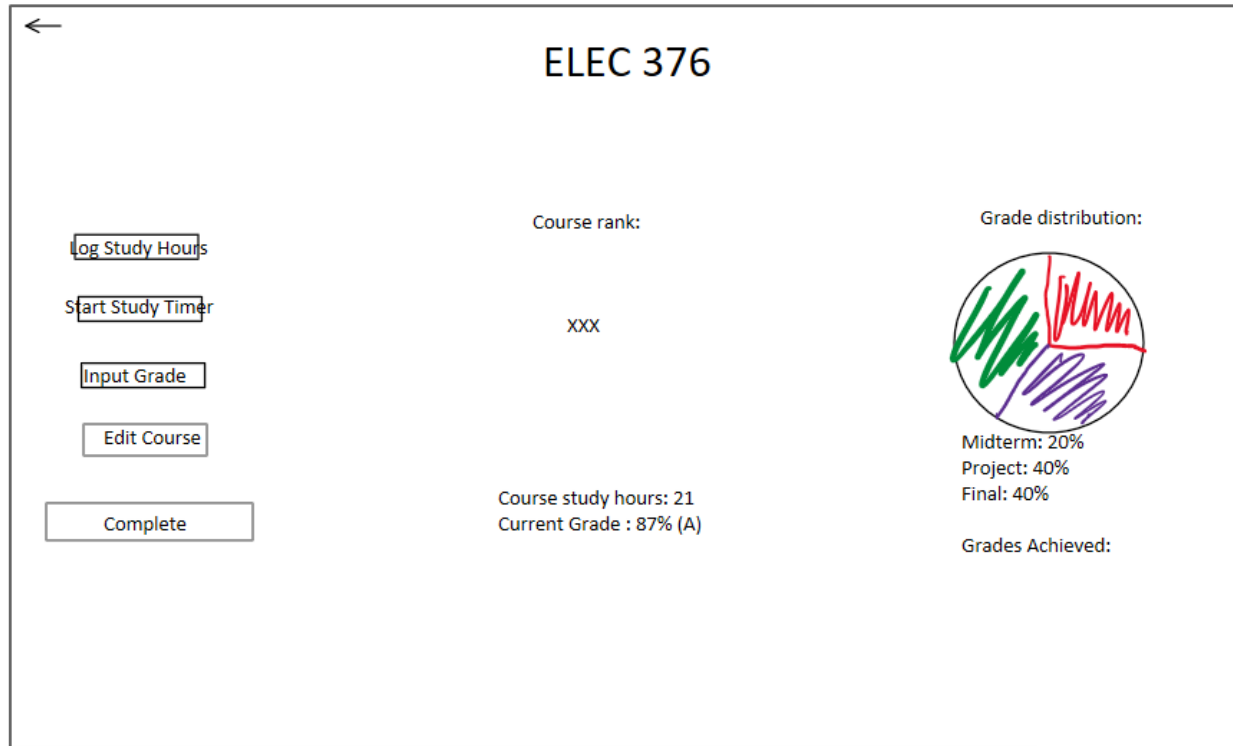


Figure 9: Window shown after clicking an individual course

From the homepage, clicking on a friend's name should bring the user to a window in which you can see that friend's rank, individual course ranks, and study hours (Figure 4). The team would like to implement this to add the feeling of friendly competition, requiring the ability to see friends' progress alongside the user's.

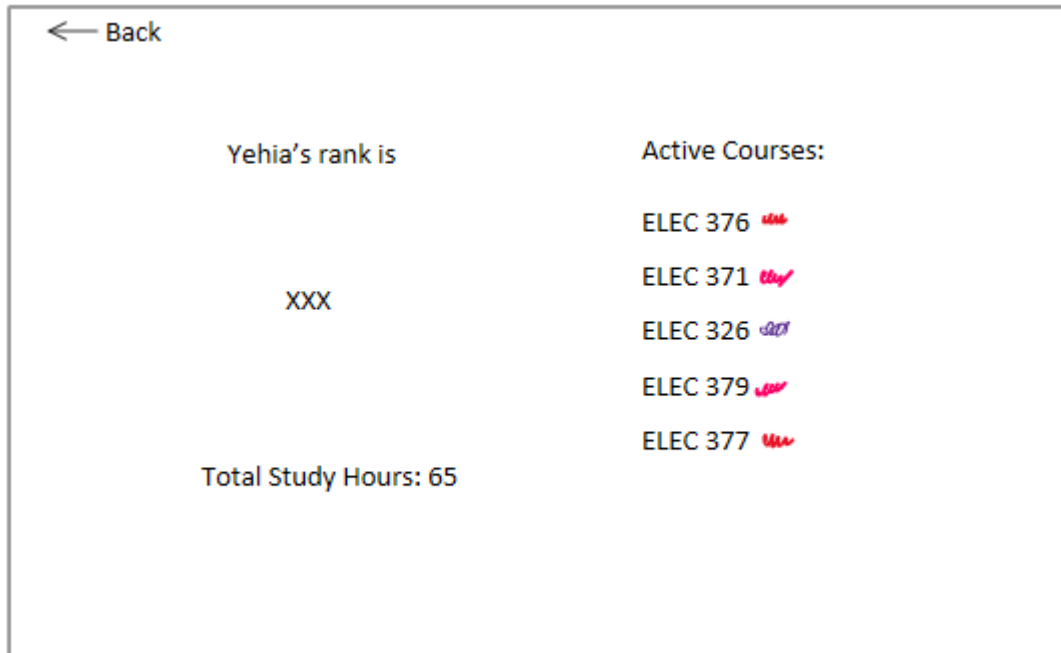


Figure 10: Window shown after clicking on a friend

Finally, the user should be able to complete courses such that old courses are not incorporated into their current grade as seen in Figure 5. Clicking on "Courses" from the homepage will bring users to a window showing all courses, grades, and course ranks, past and current.

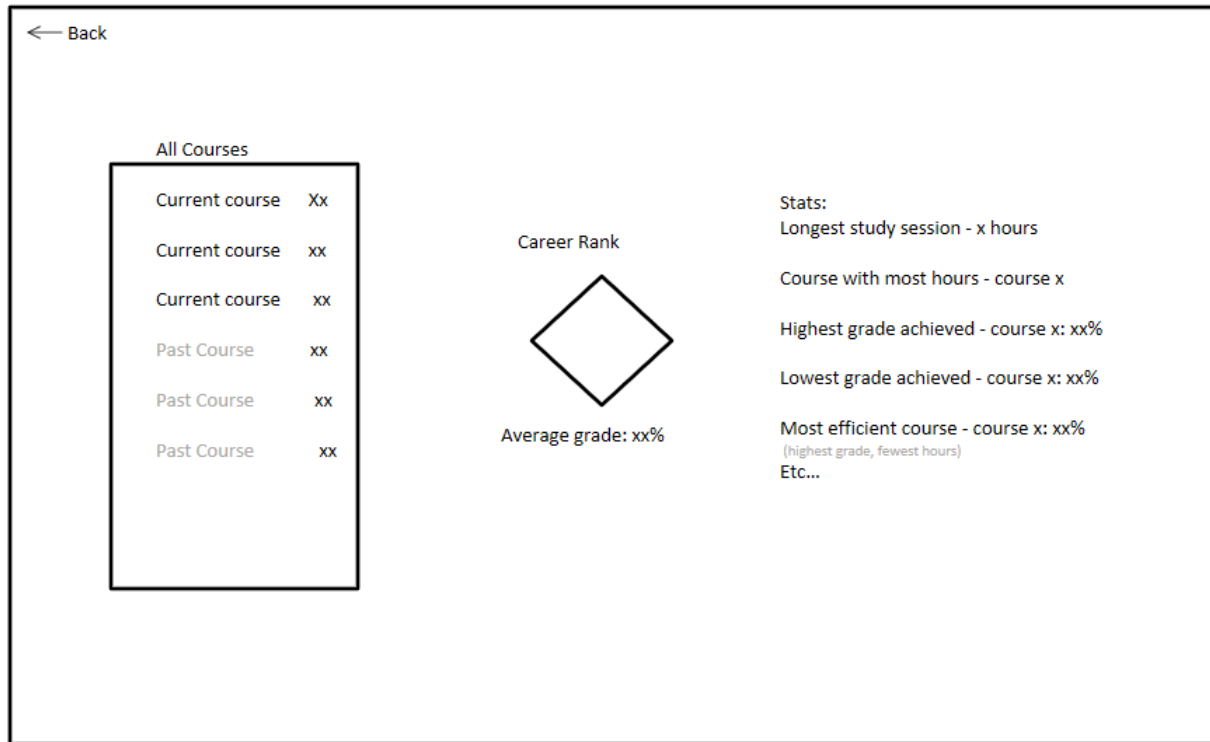


Figure 11: The window shown after clicking 'Courses' from the homepage

An interesting functionality would be to have a way for the user to see a rank which takes into consideration all courses added over the timeline of their account. If time permits, this feature will be explored along with 'Career Stats'. It is also in the interest of the team that, if implemented, these features should be visible between friends such that peers can compare their 'stats'.

13. Project Timeline: Gantt Chart

The Gantt Chart in Figure 6 shows the completed timeline for Sprint 1 and projected timeline for Sprint 2. In Sprint 1, the team focused on materializing ideas. The team made the database, drafted designs, and started on the back-end code. Goals were then set for what functionalities to implement and how these ideas would come to life. In the end, a basic GUI was created with some back-end code, but some main functionalities initially aimed for were still missing.

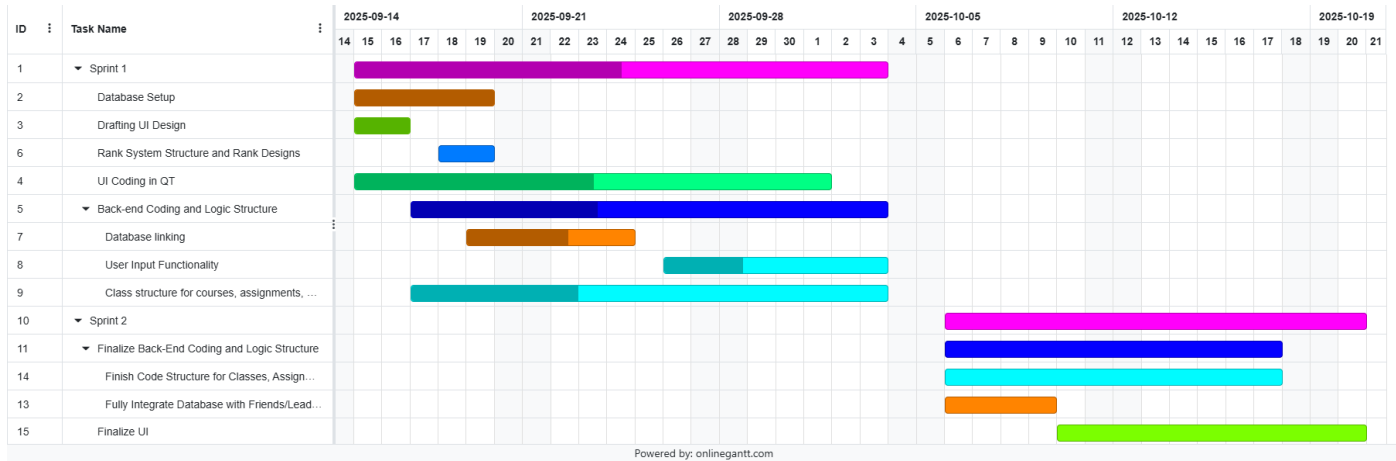


Figure 12: Gantt chart showing the completion of the project so far and projected timeline

Now, in Sprint 2, the team aims to fully implement what was imagined the application to be. There is a strong focus on fully developing the back-end code structure and logic with proper integration of the database. The final step after that is to finalize the UI design and integrate everything.